

WEEK 02

Synchronization

Theory Pack

Locks, mutexes, condition variables, deadlocks.

*Last week you broke programs on purpose.
This week you fix them.
Then you discover the new bugs that fixing
things creates.*

Welcome to deadlock.

INSIDE

- 01** Locks and Mutual Exclusion
- 02** The Pthread Mutex API
- 03** Condition Variables
- 04** Deadlock and Other New Horrors

SECTION 01

Locks and Mutual Exclusion

*Last week you wrote programs that broke on purpose. The villain was the race condition — non-atomic operations on shared data. This section introduces the tool we will use to fix every single race for the rest of the project: the **lock**.*

Recap: why the bug happened

Recall the race condition from last week. The line `counter++` compiles into three machine instructions: load, increment, store. When two threads execute this sequence concurrently on a shared counter, the hardware can interleave their instructions in such a way that both threads load the same starting value, both increment, and both store the same result — turning two increments into one.

The root problem was that a sequence of three instructions had to behave as if it were one. Concurrency, plus the absence of any mechanism to enforce that grouping, equals the bug.

What we want: a critical section

A **critical section** is a region of code that accesses shared data and must execute as if it were a single, indivisible unit. The wish we want to express in code is:

***Mutual exclusion:** only one thread at a time may execute the critical section. Any other thread that arrives at the same section must wait until the first one is done.*

This is exactly what a **lock** (also called a **mutex**, short for "mutual exclusion") gives us. A lock is a small piece of data with two operations:

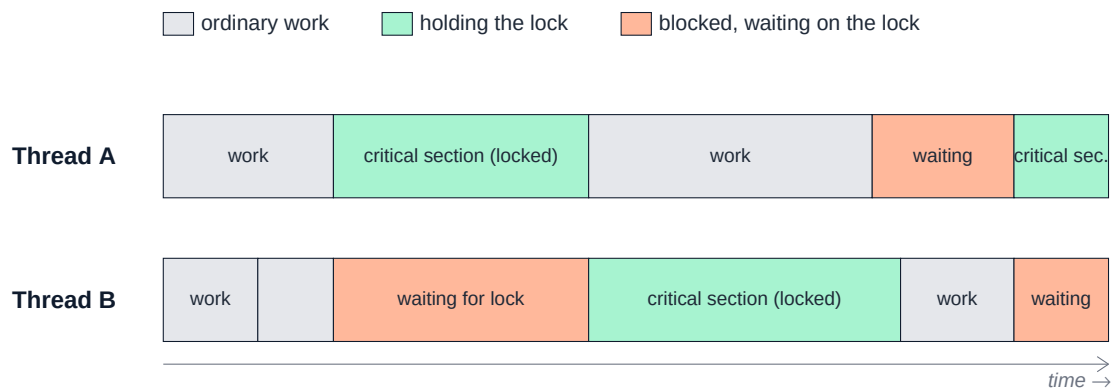
- **acquire** (lock): tries to take the lock. If another thread already holds it, this thread blocks (sleeps) until it becomes available.
- **release** (unlock): hands the lock back so the next waiting thread can acquire it.

The usage pattern is universal:

```
acquire(L);  
// critical section – shared data lives here  
release(L);
```

What it looks like in time

When two threads each run a critical section guarded by the same lock, their executions interleave only *outside* the critical section. Inside, they take strict turns — one finishes before the other can start.



Two threads share a lock. When Thread A is in its critical section, Thread B blocks; when A releases, B acquires and proceeds. The critical sections never overlap.

Properties of a good lock

Computer scientists since the 1960s have argued about what exactly a lock has to guarantee. The classical three properties:

- 1. Mutual exclusion.** At most one thread is in the critical section at any moment. This is the whole point.
- 2. Progress.** If no thread holds the lock and one or more threads want it, one of them will eventually get it. The lock does not refuse to hand itself out for no reason.
- 3. Bounded waiting.** Once a thread has decided it wants the lock, the number of times other threads can sneak in ahead of it is bounded. (In practice, this means no thread can be starved forever — though pthread mutexes do not always guarantee strict fairness, and you usually don't notice.)

How locks work, briefly

You might reasonably ask: if locking and unlocking are operations on a shared piece of data (the lock itself), why aren't *they* subject to race conditions? Aren't we just pushing the problem down one level?

The answer is: yes, and the CPU helps. Modern processors provide special hardware instructions that perform a read-modify-write in a single atomic step. The most famous is **compare-and-swap** (CAS): "atomically, if memory location X equals A, set it to B and report success; otherwise report failure." The pthreads library uses CAS (or similar primitives like test-and-set) to implement `pthread_mutex_lock` at a lower level.

For this project, you do not need to know how the mutex is implemented. You only need to trust that it works.

A preview: fixing counter++

With a lock, the racy counter from last week becomes correct:

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
long counter = 0;

void *increment(void *arg) {
    for (long i = 0; i < ITTERS; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

Now no matter how the OS schedules the two threads, only one of them can be inside `counter++` at a time. The other one must wait its turn. The final count is always exactly $2 * ITTERS$ — every single run, on every machine.

What we gained: correctness. **What we paid for it:** time. The locked version is slower than the racy one, because threads now spend time waiting for each other instead of doing work. We'll see how to manage this trade-off shortly.

SECTION 02

The Pthread Mutex API

Time for the actual code. The `pthread` library gives you a small handful of functions for creating, locking, unlocking, and destroying mutexes. You'll use these constantly for the rest of the project.

The type and how to create one

A mutex is a value of type `pthread_mutex_t`. There are two ways to create one. For mutexes that live for the entire lifetime of the program (which is almost always what you want), use the static initializer:

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
```

This works for globals and for any mutex whose default attributes you're happy with. For dynamically allocated mutexes (for example, one inside a struct that you `malloc`), use the function form:

```
pthread_mutex_t lock;
pthread_mutex_init(&lock, NULL); // NULL = default attributes
// ... use the lock ...
pthread_mutex_destroy(&lock);    // when you're done forever
```

The operations

Function	What it does
<code>pthread_mutex_lock(&m)</code>	Acquire the mutex. If another thread holds it, block until it's free.
<code>pthread_mutex_unlock(&m)</code>	Release the mutex. If other threads are waiting, one of them will be woken.
<code>pthread_mutex_trylock(&m)</code>	Try to acquire without blocking. Returns 0 on success, <code>EBUSY</code> if held.
<code>pthread_mutex_destroy(&m)</code>	Clean up. Only call this when no thread will ever use the mutex again.

The pattern, drilled in

Every critical section in every program you write this week will follow this shape. Burn it into your fingers:

```
pthread_mutex_lock(&m);  
// ... critical section: read/write shared data ...  
pthread_mutex_unlock(&m);
```

Every lock you take, you must release. If a thread returns or exits while holding a mutex, no other thread can ever acquire it, and your program hangs. Pay particular attention to early returns and error paths — they are the most common source of forgotten unlocks.

Worked example: fixing the bank

Recall last week's check-then-act race in the bank example. The two-step pattern "check the balance, then withdraw" was split by the OS at exactly the wrong moment. Here is the fix:

```
int balance = 10000;  
pthread_mutex_t balance_lock = PTHREAD_MUTEX_INITIALIZER;  
  
void *customer(void *arg) {  
    for (int i = 0; i < ATTEMPTS; i++) {  
        pthread_mutex_lock(&balance_lock);  
        if (balance >= WITHDRAWAL_AMOUNT) {  
            balance -= WITHDRAWAL_AMOUNT;  
        }  
        pthread_mutex_unlock(&balance_lock);  
    }  
    return NULL;  
}
```

Notice that the check *and* the act are both inside the critical section. If you only locked around the subtraction, the race would still happen — another thread could pass its own check before yours grabbed the lock. The whole compound operation must be atomic.

Lock granularity

"How much should one lock protect?" is one of the central design questions in concurrent programming. There are two extremes:

Coarse-grained locking: one big lock for everything. Simple. Easy to reason about — if anybody's touching the shared state, they hold the lock. The downside: it's a bottleneck. Only one thread can be doing useful work at a time, no matter how many cores you have.

Fine-grained locking: many small locks, each protecting just one variable or one entry in a data structure. More parallelism (different threads can work on different parts at the same time). Much harder to get right — you can introduce deadlocks (next section!) and your code becomes a tangle of lock and unlock calls.

Rule of thumb for this project: start coarse-grained (one lock per data structure). Only break it up if you have evidence the lock is causing a real performance bottleneck. Don't optimize prematurely. Correctness first, performance second.

Alternative: atomic operations

For simple shared variables — a counter, a flag, a single pointer — C11 provides **atomic types** in `<stdatomic.h>` that don't need a separate lock. An `atomic_int` supports increment, decrement, compare-and-swap, and other operations that are guaranteed indivisible at the hardware level.

```
#include <stdatomic.h>
atomic_long counter = 0;

void *increment(void *arg) {
    for (long i = 0; i < ITERS; i++) {
        atomic_fetch_add(&counter, 1);    // atomic, no lock needed
    }
    return NULL;
}
```

Atomics are typically much faster than mutexes for these simple cases — they map to a single CPU instruction rather than a round-trip through the kernel's lock-management machinery. But they only help you when your critical section is a single operation. Anything more complex (a check-then-act, a two-field update) still needs a mutex.

SECTION 03

Condition Variables

Locks handle the "don't step on each other" problem. But threads often need to coordinate: one thread waits until another has produced something for it to consume. This section introduces the tool for that: the condition variable.

The new problem

Suppose you have a shared queue. One thread (the producer) adds items to it. Another thread (the consumer) takes items out and processes them. With just a mutex, you can make sure they don't corrupt the queue. But what should the consumer do when the queue is empty?

A natural first attempt: just keep checking until something shows up.

```
while (queue_is_empty()) {
    // do nothing, just loop
}
process_item();
```

This is called **busy-waiting** or **spinning**, and it is terrible. It burns CPU cycles doing literally nothing useful. It also takes CPU time away from the producer thread, which is the one thread that could actually make the condition true. The consumer effectively slows down the producer it's waiting on.

What we want instead: the consumer should go to sleep until the producer wakes it up. That's exactly what a condition variable provides.

What a condition variable is

A condition variable is a queue of waiting threads, associated with a mutex. A thread can:

- **wait** on the condition variable — atomically releasing the mutex and going to sleep

- **signal** the condition variable — waking up one thread that's waiting
- **broadcast** the condition variable — waking up all threads that are waiting

Crucially, a condition variable on its own is dumb — it just manages a queue of sleepers. The actual *condition* (the queue being non-empty, the resource being available, whatever) lives in shared variables that you protect with the associated mutex. The condition variable just provides the "go to sleep until poked" mechanism.

The API

```
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;

pthread_cond_wait(&cv, &mutex);    // sleep, releasing mutex
pthread_cond_signal(&cv);          // wake one waiter
pthread_cond_broadcast(&cv);       // wake all waiters
```

The strange and important thing about `pthread_cond_wait` is its atomicity. It does *three* things, all as one indivisible step:

1. Releases the mutex.
2. Puts this thread to sleep on the condition variable's queue.
3. When woken, re-acquires the mutex before returning.

The atomicity of steps 1 and 2 is critical. If `cond_wait` released the mutex and *then* went to sleep, a signal that arrived in between would be lost — the signaler would have already woken nobody, and the waiter would then sleep forever. The API is designed specifically to make this impossible.

The canonical pattern

Every use of a condition variable follows this shape. Memorize it.

```
pthread_mutex_lock(&mutex);
while (!condition_is_true()) {
    pthread_cond_wait(&cv, &mutex);
}
// do the thing that requires condition_is_true
pthread_mutex_unlock(&mutex);
```

Why *while* and not *if*

The waiting must be in a loop, not in an `if`, for two reasons:

Spurious wakeups. The pthreads specification permits `cond_wait` to return without anyone having signaled. It's rare, but real. Always re-check the condition after waking.

Another thread got there first. Suppose two consumers are waiting and the producer adds one item, then broadcasts. Both consumers wake. One reacquires the mutex, sees the item, and takes it. By the time the second consumer reacquires the mutex, the queue is empty again. If it didn't re-check the condition, it would try to take from an empty queue and crash.

Always wait in a while loop. Always. The number of real-world bugs from someone writing `if` instead of `while` here is astonishing.

Signal vs broadcast

`signal` wakes one waiting thread. `broadcast` wakes all of them.

Use **signal** when you know that only one waiter can usefully proceed — for example, you added exactly one item to a queue, so only one consumer can take it. Use **broadcast** when multiple waiters might be able to proceed — for example, you released a shared resource that several waiters were watching for, or you've changed the state in a way that multiple kinds of waiters care about.

When in doubt, broadcast is safer (at worst, some threads wake up, recheck their condition, find it still false, and go back to sleep — a small CPU cost but correct behavior).

The classic example: producer-consumer

A bounded buffer (a queue with a fixed maximum size) with a producer adding items and a consumer removing them is the "Hello World" of condition variables. Two waiting conditions exist: the consumer waits when the buffer is empty; the producer waits when the buffer is full. So we need two condition variables, both associated with the same mutex.

Full code is in `03_producer_consumer.c`. The shape:

```
// Producer
pthread_mutex_lock(&lock);
while (count == BUFFER_SIZE) {
    pthread_cond_wait(&not_full, &lock);
}
buffer[in] = item; in = (in+1) % BUFFER_SIZE; count++;
pthread_cond_signal(&not_empty);
pthread_mutex_unlock(&lock);

// Consumer
pthread_mutex_lock(&lock);
while (count == 0) {
    pthread_cond_wait(&not_empty, &lock);
}
item = buffer[out]; out = (out+1) % BUFFER_SIZE; count--;
pthread_cond_signal(&not_full);
pthread_mutex_unlock(&lock);
```

Note the symmetry. Producer waits on `not_full` and signals `not_empty`. Consumer does the opposite. Both check their condition in a while loop. Both perform the actual work inside the critical section. This is the template for virtually every coordination pattern you'll ever write.

SECTION 04

Deadlock and Other New Horrors

You have learned to fix race conditions with locks. Congratulations — you now have access to a new class of bugs, the most famous of which is deadlock. This section is about the bugs that synchronization itself can create.

What a deadlock is

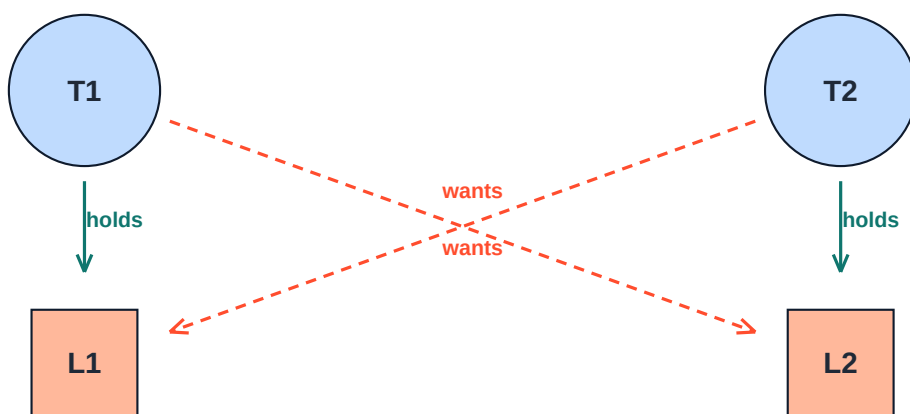
A **deadlock** is a situation where two or more threads are each waiting for a resource that another thread in the group holds, in such a way that none of them will ever be able to proceed. The program just stops. There is no error message. It simply does not make progress, ever.

The simplest example uses two threads and two locks:

```
// Thread A           // Thread B
lock(X);              lock(Y);
// ...                // ...
lock(Y); <-- waits for B lock(X); <-- waits for A
```

Thread A holds X and wants Y. Thread B holds Y and wants X. Neither will release what they hold until they get what they want — but what they want will never be released. The program is now permanently stuck.

We can visualize this as a graph: nodes are threads and locks, edges are "holds" and "wants" relationships. A deadlock corresponds exactly to a cycle in this graph.



Solid: lock currently held. Dashed: lock requested but waiting. The cycle is the deadlock.

You'll see this exact deadlock — printed line by line as it develops — in `04_deadlock_demo.c`.

The four Coffman conditions

In 1971, Edward Coffman characterized exactly when deadlock is possible. **All four** of these conditions must hold simultaneously for a deadlock to occur:

Condition	What it means
1. Mutual exclusion	Resources can only be held by one thread at a time. (Locks have this by design.)
2. Hold and wait	A thread holding one resource can request another.
3. No preemption	Resources can't be forcibly taken from a thread; only released voluntarily.
4. Circular wait	There exists a cycle of threads each waiting for the next.

Break *any one* of these and deadlock is impossible. The first three are usually inherent properties of locks — you can't easily get rid of them. The fourth, circular wait, is the one you have actual control over.

The fix: lock ordering

If every thread in your program acquires multiple locks *in the same global order*, then no cycle can form. The circular wait condition is broken by construction.

For our two-thread example: if both Thread A and Thread B agree to acquire X before Y, the deadlock is impossible. Whichever thread grabs X first will go on to grab Y; the other will simply wait for X like a well-behaved thread waiting for a single lock.

```
// Both threads – same order:
pthread_mutex_lock(&X);
pthread_mutex_lock(&Y);
// ... use both ...
pthread_mutex_unlock(&Y);
pthread_mutex_unlock(&X);
```

Project rule: if your code ever holds more than one lock at a time, document the lock-acquisition order in a comment at the top of the relevant file. Then enforce it everywhere.

Other bugs in the family

Deadlock is the famous one, but there are some siblings worth knowing about by name.

Livelock

Threads are doing work, but no useful progress is happening. Picture two people meeting in a narrow hallway, each stepping to one side, then both stepping to the other, in perfect synchrony, forever. Each is "doing something"; neither gets past the other. Livelock typically arises from clever attempts to *avoid* deadlock that backfire.

Starvation

One thread waits forever even though the system is making progress. Other threads keep getting the resource it wants. Not a deadlock — the system isn't frozen — but unfair. A long-running, contended lock can starve waiters that always lose the race to whoever happens to be the most recently woken thread.

Priority inversion

A low-priority thread holds a lock that a high-priority thread needs. The high-priority thread is now blocked on the low-priority one — and worse, medium-priority threads can preempt the low-priority lock-holder, effectively making the high-priority thread wait for them too. This caused the Mars Pathfinder mission to keep rebooting in 1997.

Defensive concurrency

Concurrent code is hard to test. Bugs are non-deterministic and may show up only under load you can't easily reproduce. The best defense is reasoning, not testing. A few principles that will save you:

Keep critical sections small. Less code inside the lock means less contention and fewer places to make mistakes.

Never call into untrusted code while holding a lock. If the code you call also takes a lock, you might deadlock. If it calls back into your code, you might recursively try to acquire a lock you already hold.

Always release locks in the reverse order of acquisition. And always release them. Including in error paths.

Prefer fewer locks to more. One coarse lock is easier to reason about than many fine ones, and is almost always fast enough.

Document your lock invariants. A comment near the data saying "protected by foo_lock" pays for itself within a week.

***You now have the full toolkit for writing correct concurrent programs.** Head to the code examples in `code/`, then tackle the problems in `problem-statement.pdf`. Next week we use these tools to build a real parallel program with measurable speedup.*